Assignment 4.2 - Evaluating AI-Assisted Development Practices and Applying AI to Real Code

Professor: Richard Krasso
Student: Jonathan Cantu
Date: September 5, 2026

Part I: RiverPoint Systems Analysis

1. Identify Opportunities for AI-Assisted Support

RiverPoint Systems has several areas where AI-assisted tools could provide practical value. The most immediate opportunities are coding support, documentation, and test-case generation. I would not view AI as replacing any of these development activities, but rather as providing a strong first pass that developers can review and refine.

For coding, AI could help generate boilerplate, explain unfamiliar code, suggest refactoring opportunities, or propose straightforward implementations. This is particularly useful for repetitive work where the developer already understands the underlying problem. Krasso (2026) explains that AI-assisted development is most effective when it reduces repetitive or cognitively heavy work so developers can focus on higher-level decisions.

Documentation is another clear opportunity because RiverPoint already has inconsistent documentation. AI can generate initial function descriptions, API documentation, comments, and explanations of complex logic. Developers can then validate and refine those drafts rather than starting from scratch. Krasso (2026) notes that this can help prevent documentation from falling behind while still requiring human review for accuracy.

AI-generated test cases could also help address RiverPoint's inconsistent test coverage. AI can quickly brainstorm boundary conditions, unusual inputs, and possible failure scenarios that developers might overlook. This does not replace a testing strategy, but it can help broaden the initial set of scenarios considered by the team.

2. Identify Risks or Misuses of AI

The biggest risk I see is over-reliance on AI-generated code simply because it looks correct. AI can generate syntactically valid and convincing code while still misunderstanding the requirement, introducing subtle logic problems, or following patterns that do not fit the existing architecture. Krasso (2026) specifically warns that AI-generated code may appear plausible while still being logically flawed or inconsistent with team standards.

That risk is particularly important for RiverPoint because its applications support financial reporting. A calculation can compile, pass a few basic tests, and still produce the wrong business result. AI does not know whether a financial rule, validation requirement, rounding policy, or exception condition is correct unless that context is explicitly provided and then verified.

There is also risk at the opposite extreme. Developers who refuse to use AI at all may spend unnecessary time on repetitive tasks where the tool could improve productivity. The goal should therefore not be either complete trust or complete avoidance.

3. Recommend Practical Guidelines

I would recommend that RiverPoint establish a common AI-assisted development policy rather than leaving usage entirely up to individual developers.

First, developers should understand the problem before asking AI to solve it. AI should help implement a solution, not substitute for understanding the requirement. Second, all AI-generated code should go through the same review, testing, security, and quality checks as human-written code. Krasso (2026) compares AI-generated output to work produced by a junior developer: potentially useful, but still requiring careful review and validation.

I would also encourage teams to use AI primarily in areas where it provides clear leverage:

- Boilerplate and repetitive coding
- Initial documentation drafts
- Code explanations
- Test-case brainstorming
- Pull-request summaries
- Initial analysis of errors or pipeline failures

More consequential decisions—business rules, architecture, security decisions, financial calculations, and production-impacting changes—should remain under explicit human control.

Finally, AI output should always be verified against requirements and team standards. The useful question is not simply, "Did the AI produce an answer?" but, "Is the answer correct, maintainable, appropriate, and consistent with the application?"

4. Connect to the Three Ways

Used appropriately, AI-assisted development can strengthen all Three Ways.

Flow improves when developers spend less time on repetitive work. Generating a first draft of documentation, boilerplate code, or test cases can reduce friction and allow developers to focus more time on meaningful engineering problems. Krasso (2026) identifies this reduction in routine cognitive load as one of the primary benefits of AI-assisted development.

Feedback improves because AI can help identify potential bugs, missing tests, unclear documentation, or suspicious logic earlier in the development process. AI-generated test cases can also expand the scenarios considered before code reaches later stages of testing. The Week 4 reading notes that these tools can identify edge cases and possible failure points, but developers still determine whether those cases actually match expected system behavior.

Continuous Learning improves when AI helps developers understand unfamiliar code, explain patterns, and explore alternatives. However, learning only occurs if developers remain engaged with the output. If they simply accept generated code without understanding it, AI could actually work against Continuous Learning.

Ultimately, I would recommend that RiverPoint treat AI as a collaborator rather than an autopilot. The company should encourage its use where it reduces repetitive work and improves feedback, while establishing clear expectations that developers remain responsible for correctness, business context, and final decisions. That balanced approach gives RiverPoint the productivity benefits of AI without sacrificing the engineering judgment needed to maintain reliable financial applications.

Reference:

Krasso, Richard (2026). "WEB 430 Week 4 readings". Bellevue University.

Part II: AI-Assisted Development Exercise

For this exercise, I used an AI-assisted tool to review the provided Python function and generate documentation, suggest improvements, and generate test cases.

Original code:

python:

```
def calculate_total(items):
    total = 0

    for item in items:

        if "price" in item and "quantity" in item:
            total += item["price"] * item["quantity"]

        return total
```

1. AI-Generated Documentation

Calculates a total value from a collection of item dictionaries:
`calculate_total(items)`

Each item is expected to contain a `price` and `quantity`. When both keys are present, the function multiplies the item's price by its quantity and adds the result to the running total.

Parameters:

`items` – An iterable containing dictionaries representing individual items. Each valid item should contain `price` and `quantity` values.

Returns:

A numeric value representing the calculated total.

Important observation:

In the supplied implementation, `return total` is indented inside the `for` loop. As a result, the function returns after processing only the first item rather than processing the entire collection.

2. AI-Generated Code Improvement

A primary improvement is to move the `return total` statement outside the loop so every item is processed. Input validation can also make the function more predictable by ensuring that required values exist and are numeric.

python:

```
def calculate_total(items):
    """Calculate the total cost of valid items.

    Each item must contain numeric 'price' and 'quantity' values.
    Items missing either value are ignored.

    Args:
        items: An iterable of dictionaries containing price and quantity.

    Returns:
        The sum of price multiplied by quantity for all valid items.
    """

    total = 0

    for item in items:
        if "price" not in item or "quantity" not in item:
            continue

        price = item["price"]
        quantity = item["quantity"]

        if not isinstance(price, (int, float)) or not isinstance(quantity, (int,
float)):
            continue

        total += price * quantity

    return total
```

Potential additional considerations include defining how negative prices or quantities should be handled and determining whether financial calculations require a specific rounding or decimal-handling strategy. Those decisions depend on RiverPoint's business requirements rather than something the AI can determine from this function alone.

3. AI-Generated Test Cases

1. Normal single item
   Input: `[{"price": 10, "quantity": 2}]`
   Expected output: `20`

2. Multiple valid items

Input: `[{"price": 10, "quantity": 2}, {"price": 5, "quantity": 3}]`
Expected output: `35`

3. Empty list
Input: `[]`
Expected output: `0`

4. Item missing price
Input: `[{"quantity": 2}]`
Expected output: `0`

5. Item missing quantity
Input: `[{"price": 10}]`
Expected output: `0`

6. Mixed valid and incomplete items
Input: `[{"price": 10, "quantity": 2}, {"price": 5}, {"price": 3, "quantity": 4}]`
Expected output: `32`

7. Decimal price
Input: `[{"price": 9.99, "quantity": 2}]`
Expected output: `19.98`

8. Zero quantity
Input: `[{"price": 10, "quantity": 0}]`
Expected output: `0`

9. Negative quantity
Input: `[{"price": 10, "quantity": -2}]`
Expected output based on the current calculation: `-20`
Business requirement should determine whether this should instead be rejected.

10. Non-numeric price
Input: `[{"price": "10", "quantity": 2}]`
Expected output using the improved version: `0`

4. Reflection

I thought the AI did a good job of identifying the most important problem in the original code, which was that the `return` statement was inside the loop and caused the function to stop after processing the first item. It also provided useful documentation that explained both what the function was intended to do and the behavior of the code as written.

The suggested input validation was practical because it prevents unexpected values from being processed without making the function unnecessarily complicated. I also liked that the generated test cases included multiple items because that case would immediately expose the misplaced `return` statement. The cases involving missing values, decimals, zero quantities, and nonnumeric data also provided useful additional coverage. One limitation was that the AI could not determine the actual business rules for negative prices or quantities. It could identify those cases as potentially questionable, but a developer or Business Analyst would still need to determine the correct expected behavior. I would also want the team to decide how monetary values should be represented and rounded rather than allowing the AI to make that decision automatically.

Overall, the suggestions were practical because they improved correctness, clarity, documentation, and test coverage without radically redesigning a simple function. This exercise demonstrates how AI-assisted development can quickly provide a useful first review, while the developer remains responsible for validating the assumptions and deciding which recommendations actually belong in the application.